Design Diagrams:

The data structure of the memory system uses a linked list of block nodes, where each block tracks a section of the 1024-byte core memory. Each block stores a start address, a size, a free flag, and the memory handle of the task that owns it. The blocks are linked together in order by address, starting from a head pointer in the MMU. Alongside the block list, the MMU also keeps a vector of MemInfo records, one for every active allocation. Each MemInfo stores the base address, the limit, the current read pointer, and the current write pointer for that allocation. The TCB from Phase 1 was updated to also hold a memory handle so each task knows which allocation belongs to it.

The MMU class has an associative relationship with the Scheduler; the Scheduler calls the MMU to allocate memory when a new task is created. The MMU has a composition relationship with the Block linked list and the MemInfo vector, since the MMU creates and deletes these structures directly. The TCB has an associative relationship with the MMU through the memory handle, which points to the task's MemInfo entry. The Semaphore from Phase 1 is reused as a memory lock to protect the block list and the core memory from concurrent access.

Phase 3 Data Structures

```
+-------------------------------------------------------------+
|              Core Memory Array (1024 bytes)                 |
+--------+--------+--------+--------------------------------+
|  Used  |  Used  |  Used  |        Free  (........)        |
|  0-63  | 64-127 |128-191 |            192-1023            |
+--------+--------+--------+--------------------------------+

 Head
  |
  v
+---------+   +---------+   +---------+   +-----------+
| Block 0 |-->| Block 1 |-->| Block 2 |-->| Block 3   |--> NULL
| start:0 |   | start:64|   |start:128|   | start:192 |
| size:64 |   | size:64 |   | size:64 |   | size:832  |
| free:F  |   | free:F  |   | free:F  |   | free:T    |
| hdl:0   |   | hdl:1   |   | hdl:2   |   | hdl:-1    |
+---------+   +---------+   +---------+   +-----------+

 MemInfo Vector (indexed by handle):
+--------------+  +--------------+  +--------------+
| handle 0     |  | handle 1     |  | handle 2     |
| base:    0   |  | base:   64   |  | base:  128   |
| limit:  64   |  | limit:  64   |  | limit:  64   |
| read_off: 0  |  | read_off: 0  |  | read_off: 0  |
```

```
| write_off:0  |  | write_off:0  |  | write_off:0  |
| active:true  |  | active:true  |  | active:true  |
+--------------+  +--------------+  +--------------+
```

Each Block describes one section of memory, whether it is free or used, and points to the next block in the list. The MemInfo entries act as base and limit registers: they let the MMU figure out exactly where a task's memory starts and ends, and they track the current read and write position so a task can read and write one character at a time without overwriting itself. The core memory array is filled with '.' dots when it is free and '#' when it has just been freed, so a memory dump visually shows the difference between untouched memory and recently-freed memory.

Class Hierarchy

```
+----------+      +-------------+      +------------+
|   IPC    |----->|  Scheduler  |<>----|    TCB     |
+----------+ uses +-------------+ 1..* +------------+
| -sched   |      | -head       |      | -task_id   |
| +Send    |      | -current    |      | -task_name |
| +Receive |      | -next_id    |      | -mem_handle| (NEW)
| +Count   |      | -mem        |(NEW) | -state     |
| +Print   |      | +create_task|      | -next      |
| +Delete  |      | +yield      |      | -mailbox   |
| +Dump    |      | +dump       |      | -mailbox   |
+----------+      | +get_by_id  |      |   semaphore|
                  | +get_by_    |      +------------+
                  |   handle    |(NEW)        |
                  +-------------+             | 1..*
                         |                    v
                         | uses         +-----------+
                         v              |  Message  |
                  +-------------+       +-----------+
                  |     MMU     | (NEW) | -source   |
                  +-------------+       | -dest     |
                  | -memory     |       | -arrival  |
                  | -size       |       | -type     |
                  | -page_size  |       | -size     |
                  | -head       |       | -text     |
                  | -mem_info   |       +-----------+
                  | -memory_sem |
                  +-------------+       +-----------+
                  | +Mem_Alloc  |--<>-->|   Block   | (NEW)
                  | +Mem_Free   |  1..* +-----------+
                  | +Mem_Read   |       | -start    |
                  | +Mem_Write  |       | -size     |
                  | +Mem_Coal.  |       | -free     |
                  | +Mem_Dump   |       | -mem_hdl  |
                  | +Print      |       | -next     |
                  | +Print_Info |       +-----------+
                  +-------------+
                         |               +-----------+
                         +------<>------>|  MemInfo  | (NEW)
```

```
1..*  +-----------+
      | -base     |
      | -limit    |
      | -read_off |
      | -write_off|
      | -active   |
      +-----------+
```

The IPC class still uses the Scheduler to find destination tasks, just like in Phase 2. The Scheduler now also uses the MMU to allocate memory for each new task it creates. The MMU owns both the Block linked list and the MemInfo vector, so it controls their entire lifecycle. The TCB holds a memory handle that links the task to its entry in the MemInfo vector. The Semaphore class is used in two places: each TCB uses one to lock its mailbox, and the MMU uses one to lock memory operations.